That's the right approach. Instead of following a time-based plan, treat this as an **engineering roadmap**. Do not move to the next phase until the current one is fully working and tested. By the end, you'll have built the complete WikiLLM system yourself and understand every component rather than just assembling libraries.

The proposal describes the following high-level architecture: document upload → parsing → knowledge extraction → wiki generation → structured storage → retrieval → LLM question answering. Your learning path should follow this exact order.

---

# PHASE 1 — Understand the Entire Project

## Goal

Before writing any code, understand exactly what you are building.

Your objective is **not** to build a chatbot.

You are building an AI Knowledge Management System that converts documents into structured knowledge similar to Wikipedia and then answers questions using that knowledge.

---

## What you will learn

- What WikiLLM is
- Traditional RAG vs WikiLLM
- Complete project architecture
- How data flows through the system
- Why every component exists

---

## Learn from AntiGravity

Search topics like:

- What is RAG?
- What are embeddings?
- What is LangChain?
- What is LangGraph?
- What is FastAPI?
- What is ChromaDB?

- What is Markdown Knowledge Base?
- What is Knowledge Graph?
- What is Entity Extraction?
- What is Prompt Engineering?

Don't start coding.

Just understand.

---

# Deliverable

You should be able to draw this architecture from memory.

```
Upload

↓

Parse

↓

Knowledge Extraction

↓

Wiki Generation

↓

Storage

↓

Embeddings

↓

Retriever

↓

LLM

↓

Answer
```

---

# Testing

If someone asks

"What happens after PDF upload?"

You should explain every step without looking at notes.

Only then move forward.

---

# PHASE 2 — Development Environment

## Goal

Prepare a professional development environment.

---

## Learn

- Python Virtual Environment
- Pip
- Git
- VS Code
- Project structure
- Environment variables
- Dependency management

---

## Build

Create

```
wikillm/

backend/

frontend/

docs/

wiki/

storage/
```

```
tests/
```

Create

```
.gitignore
```

```
.env
```

```
README.md
```

```
requirements.txt
```

Install every dependency one by one.

Learn what every package does.

Never install packages you don't understand.

---

## Learn from AntiGravity

- Python virtual environment
- requirements.txt
- Git basics
- Folder architecture
- dotenv

---

## Testing

Verify:

- Python runs
- Virtual environment works
- Git repository initialized
- FastAPI server starts
- Dependencies install correctly
- Environment variables load successfully

---

# PHASE 3 — Backend Foundation

# Goal

Build the API before implementing AI.

---

# Learn

- FastAPI
- REST APIs
- Request/Response
- File Upload
- API Documentation
- Error Handling

---

# Build

Create APIs like:

```
GET /

POST /upload

GET /health
```

Don't process files yet.

Just receive them.

---

# Learn from AntiGravity

- FastAPI Tutorial
- UploadFile
- Pydantic
- API Routing
- Dependency Injection

---

# Testing

Test using Swagger and Postman.

Verify:

- Server starts
- Upload endpoint accepts PDF
- Invalid files rejected
- Large files accepted
- Error messages returned correctly

---

# PHASE 4 — Document Upload System

## Goal

Store uploaded documents safely.

---

## Learn

- File handling
- File validation
- UUID filenames
- File storage strategies

---

## Build

Accept

- PDF
- DOCX
- TXT

Store them in

`documents/`

Keep metadata.

---

## Learn from AntiGravity

- Python File Handling
- UploadFile
- pathlib
- shutil

---

## Testing

Test

- 1 PDF
- 10 PDFs
- Large PDF
- Empty PDF
- Wrong extension
- Duplicate filename

Verify

- File saved
- Correct filename
- Metadata stored
- No overwrite

---

# PHASE 5 — Document Parsing

## Goal

Extract readable text from documents.

The proposal specifies using PyMuPDF for PDF parsing.

---

## Learn

- PyMuPDF
- DOCX parsing

- TXT reading
- Text extraction

---

## Build

Convert every document into plain text.

Keep page order.

---

## Learn from AntiGravity

- PyMuPDF
- fitz
- python-docx

---

## Testing

Test

- Simple PDF
- Multi-page PDF
- Images inside PDF
- Tables
- Large document

Verify

- No missing pages
- Correct order
- Text readable
- No corruption

---

# PHASE 6 — Text Cleaning

## Goal

Prepare clean text for AI.

---

# Learn

- Regular Expressions
- Text preprocessing
- Unicode
- Whitespace normalization

---

# Build

Remove

- Extra spaces
- Blank lines
- Headers
- Footers
- Page numbers

---

# Learn from AntiGravity

- Python regex
- Text preprocessing

---

# Testing

Compare

Original PDF

↓

Extracted text

↓

Cleaned text

Ensure nothing important is removed.

---

# PHASE 7 — Knowledge Extraction

This is the core AI phase.

The proposal requires extracting entities, concepts, facts, definitions, summaries, and relationships into structured JSON.

---

## Goal

Teach the LLM to convert text into structured knowledge.

---

## Learn

- OpenAI API
- Structured Outputs
- JSON Schema
- Prompt Engineering
- Entity Extraction
- Relationship Extraction

---

## Build

Input

```
Tesla was founded by Elon Musk.
```

Output

```
Entity

Facts

Relationships
```

Summary

Store as JSON.

---

## Learn from AntiGravity

- Prompt Engineering
- Structured Output
- Function Calling
- JSON Response

---

## Testing

Check

- Correct entities
- Correct relationships
- Correct facts
- No hallucinations
- Consistent JSON structure

Repeat on many documents.

---

# PHASE 8 — Wiki Generation

The proposal transforms extracted knowledge into wiki-style pages.

---

## Goal

Generate Wikipedia-like pages automatically.

---

## Learn

- Markdown
- Wiki Links
- Knowledge Organization

---

## Build

Generate

```
Tesla.md

Elon Musk.md

Electric Vehicles.md
```

Create internal links.

---

## Learn from AntiGravity

- Markdown
- Wiki syntax
- Knowledge organization

---

## Testing

Verify

- One page per entity
- Internal links work
- No duplicate pages
- Human-readable pages

---

# PHASE 9 — Knowledge Storage

The proposal recommends Markdown, JSON, and SQLite/PostgreSQL metadata.

---

# Goal

Persist the generated knowledge.

---

# Learn

- SQLAlchemy
- SQLite
- PostgreSQL
- File storage

---

# Build

Store

- Documents
- Entities
- Relationships
- Metadata
- Wiki pages
- JSON

---

# Learn from AntiGravity

- SQLAlchemy ORM
- SQLite
- PostgreSQL basics

---

# Testing

Verify

- Database tables created
- Records inserted
- Relationships preserved
- Reload works after restart

---

# PHASE 10 — Embeddings & Semantic Search

The architecture optionally uses ChromaDB/FAISS for semantic search.

---

## Goal

Enable semantic retrieval of wiki pages.

---

## Learn

- Embeddings
- Vector databases
- ChromaDB
- Similarity search

---

## Build

Embed every wiki page.

Store vectors.

Search by meaning.

---

## Learn from AntiGravity

- Embeddings
- ChromaDB
- Vector Search

---

## Testing

Ask

- EV manufacturers
- Tesla founder
- Battery technology

Verify relevant pages are returned.

---

# PHASE 11 — Intelligent Retrieval

## Goal

Retrieve the best knowledge before asking the LLM.

---

## Learn

- Retrieval strategies
- Multi-hop retrieval
- Context building

---

## Build

Question

↓

Relevant wiki pages

↓

Related pages

↓

Final context

---

## Learn from AntiGravity

- Retriever
- Multi-hop retrieval
- Context assembly

---

## Testing

Verify

- Related pages included
- Context not too large
- Relevant information prioritized

---

# PHASE 12 — Question Answering

The proposal describes retrieving wiki pages, collecting linked pages, sending context to the LLM, and generating the answer.

---

## Goal

Generate answers only from the stored knowledge.

---

## Learn

- Prompt design
- Context injection
- Citation strategies

---

## Build

Question

↓

Retriever

↓

Context

↓

LLM

↓

Answer

---

## Learn from AntiGravity

- QA systems
- Grounded generation
- Citation prompting

---

## Testing

Test

- Direct questions
- Comparison questions
- Cross-document reasoning
- Unknown questions

Verify the model refuses to invent answers when knowledge is missing.

---

# PHASE 13 — LangChain

## Goal

Replace standalone scripts with reusable components.

---

## Learn

- Chains
- Prompt Templates
- Output Parsers
- Document Loaders

---

## Build

Refactor individual modules into LangChain components.

---

## Learn from AntiGravity

- LangChain fundamentals
- Chains
- Output parsers

---

## Testing

Ensure results are identical to the earlier implementation.

---

# PHASE 14 — LangGraph

The proposal identifies LangGraph as the orchestration layer.

---

## Goal

Convert the entire workflow into an AI graph.

---

# Learn

- Nodes
- Edges
- State
- Conditional routing

---

# Build

```
Upload

↓

Parse

↓

Clean

↓

Extract

↓

Generate Wiki

↓

Store

↓

Embed

↓

Complete
```

### Question flow

```
Question

↓

Retrieve

↓
```

```
Context

↓

LLM

↓

Answer
```

---

## Learn from AntiGravity

- LangGraph StateGraph
- Nodes
- State management
- Conditional edges

---

## Testing

Verify every node executes correctly.

Test failure recovery.

Test repeated execution.

---

# PHASE 15 — Frontend

## Goal

Create the user interface.

---

## Learn

- React
- API integration
- File uploads
- Chat interface

---

## Build

Pages

- Dashboard
- Upload
- Wiki Browser
- Chat
- Document list

---

## Learn from AntiGravity

- React fundamentals
- Axios
- File upload
- React Router

---

## Testing

Verify

- Upload works
- Chat works
- Wiki pages display
- API integration successful

---

# PHASE 16 — Complete System Testing

This phase validates the full project from end to end.

## Functional Testing

- Upload one document
- Upload multiple documents
- PDF, DOCX, TXT support

- Duplicate uploads
- Invalid file rejection

# Parsing Testing

- Long PDFs
- Tables
- Empty pages
- Mixed formatting

# Knowledge Extraction Testing

- Correct entities
- Correct facts
- Correct relationships
- Stable JSON output

# Wiki Generation Testing

- One page per entity
- Correct links
- No duplicates

# Storage Testing

- Database persistence
- Markdown generation
- JSON generation

# Retrieval Testing

- Relevant page retrieval
- Multi-hop retrieval
- Cross-document retrieval

# Question Answering Testing

- Simple factual questions
- Relationship questions
- Multi-step reasoning
- Questions with no answer

# Performance Testing

Measure

- Upload time
- Parsing time
- Extraction time
- Wiki generation time
- Embedding time
- Retrieval latency
- Total response time

## Error Handling Testing

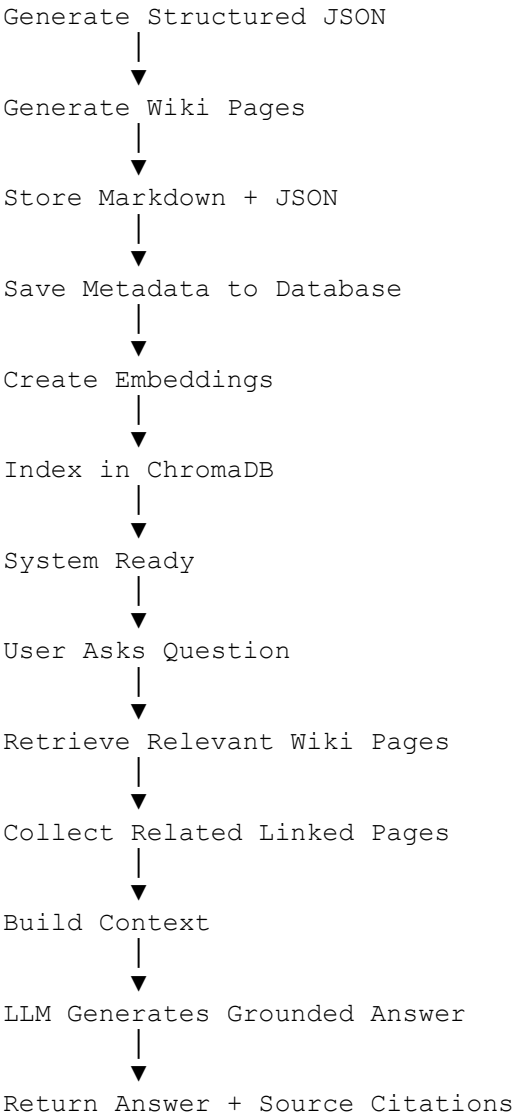
- Corrupted files
- Empty documents
- API failures
- Database unavailable
- LLM timeout
- Invalid user input

---

# Final End-to-End Run

When every phase passes its tests, your complete execution flow should be:

```
Start FastAPI Server
        |
        ▼
Open Web Application
        |
        ▼
Upload PDF/DOCX/TXT
        |
        ▼
Validate File
        |
        ▼
Store Original Document
        |
        ▼
Parse Document
        |
        ▼
Clean Extracted Text
        |
        ▼
LLM Knowledge Extraction
        |
        ▼
```




This roadmap follows the architecture and workflow described in your proposal while breaking it into self-contained engineering phases. Completing each phase, learning the underlying concepts through AntiGravity, and passing the listed tests before proceeding will leave you with a fully working WikiLLM knowledge management platform that you understand end to end.